

Лабораторна робота 04

MCP-сервер, мультиагентна конфігурація та оцінювання

Курс «Прикладні ШІ-агенти та системи на основі великих мовних моделей» · 2026/2027

Лекції	13, 16, 17
Аудиторний час	4 год; самостійна робота 8 год
Формат	індивідуально
Вартість	0 грн: локальна модель або stub, платні API не потрібні

Мета

Опублікувати власний MCP-сервер лише для читання, порівняти одиночного агента з мультиагентною конфігурацією на одному наборі задач і показати числами, чи виправдана друга архітектура.

Теоретичні відомості

Доки кожен застосунок писав власну інтеграцію до кожного джерела даних, кількість з'єднань зростала як $M \times N$. Протокол зводить її до $M + N$: застосунок реалізує клієнтську частину один раз, джерело — серверну. Model Context Protocol описує архітектуру host, client і server, обмін у форматі JSON-RPC і три примітиви: інструменти, ресурси і промпти [1]. Локальний сервер працює через транспорт stdio і не потребує ані мережі, ані облікових даних.

Протокол не є межею довіри. Сервер MCP — це стороння програма, що виконується з вашими правами, тому перевіряють походження сервера, обсяг наданих прав і наявність аудиту, а за замовчуванням дають лише читання [7].

Другий агент множить не моделі, а контексти, набори прав і критерії зупинки. Опубліковані дані розходяться. Anthropic повідомляє про перевагу мультиагентної дослідницької системи над одиночним агентом на внутрішньому наборі і водночас про кратне зростання витрат токенів [3]. Cognition аргументує протилежне: розділення контексту породжує неявні рішення, які агенти не бачать один в одного, і надійніше будувати одного сильного агента [4]. Розбір анотованих трас мультиагентних систем дає таксономію відмов, у якій помітну частку становлять порушення специфікації, розсинхронізація між агентами і неправильне завершення задачі [5]. Практичний висновок один: сильний одиночний агент із кращими інструментами є базовою лінією, яку треба перемогти, а не пропустити.

Перемогу доводять вимірюванням. Одна агрегована цифра нічого не діагностує, тому розділяють результат, шлях до нього і ціну: чи розв'язана задача, чи правильно обрані інструменти й аргументи, чи підтверджені твердження джерелами, скільки токенів і часу це коштувало [6]. Для надійності важлива не лише частка успіху, а її повторюваність: метрика pass^k вимагає успіху в усіх k незалежних прогонах тієї самої задачі, тоді як $\text{pass}@k$ задовольняється хоча б одним успіхом.

Різницю між конфігураціями оцінюють з довірчим інтервалом. Для частки $\hat{p} = x/n$ інтервал Вілсона

$$\frac{\hat{p} + \frac{z^2}{2n} \pm z \sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}$$

де n — кількість задач, $z = 1.96$ для рівня 95%, поводить коректно і на малих вибірках. Але інтервал Вілсона описує кожну частку окремо і не є тестом різниці: висновок «інтервали перекриваються, отже конфігурації однакові» некоректний. Різницю оцінюють парно

— обидві конфігурації міряються на тому самому наборі, для кожної задачі береться різниця результатів, і 95 % інтервал будується вже для середньої різниці (парний bootstrap). Три прогони тих самих сорока задач не дають ста двадцяти незалежних спостережень: одиниця ресемплінгу — задача, а прогони всередині неї усереднюються або зводяться в `pass`³. На наборі з сорока задач різниця в дві-три задачі майже напевно лежить усередині інтервалу різниці і не є доказом.

Там, де еталонної відповіді немає, оцінку доручають іншій моделі. Такий суддя має власні систематичні зсуви — позиційний, схильність до довших відповідей і до власних текстів [2]. Тому суддю калібрують: беруть підвибірку з людськими мітками і рахують узгодженість, наприклад через каппу Коена. Некалібрований суддя вимірює власні уподобання.

Кожен прогін лишає трасу. Семантичні конвенції OpenTelemetry для генеративних систем фіксують імена атрибутів для системи, операції та спожитих токенів, що дозволяє збирати траси різних інструментів в одному сховищі [6].

Хід роботи

Позначка *ауд.* — крок виконується на занятті, *сам.* — самостійно в межах СРС. Складання набору задач, 240 агентних прогонів і калібрування судді свідомо винесені на домашній етап. Аудиторна частина розрахована на 4 години.

1. **МСР-сервер (60 хв, ауд.).** Два інструменти лише для читання над локальними даними. *Перевірка:* у `stdio`-режимі жоден `print()` не йде в `stdout` — це найчастіший баг, і він ламає протокол мовчки. Логи тільки в `stderr`.
2. **Клієнт і аудит-лог (40 хв, ауд.).** Отримати перелік інструментів, викликати їх. *Перевірка:* у логу є час, інструмент, аргументи, результат і тривалість кожного виклику.
3. **Тести схем (30 хв, ауд.).** Два різні рівні. Відсутнє обов'язкове поле і поле неправильного типу не проходять схему — це протокольна помилка JSON-RPC. Семантично хибні дані (неіснуюча дата, невідомий ідентифікатор) схему проходять і повертаються як результат із `isError: true`, придатний для самовиправлення моделі. *Перевірка:* у тестах є обидва випадки і вони дають різні форми.
4. **Набір задач (50 хв, сам.).** 40 задач із відомими відповідями, **фіксується до експериментів.**
5. **Дві конфігурації (60 хв, ауд.).** Одиночний агент і `supervisor` із двома виконавцями. *Однакові інструменти, однакова модель* — інакше порівнюєте не те.
6. **Прогін (40 хв, сам.).** Три прогони кожної конфігурації на всіх 40 задачах. *Перевірка:* рахуйте не лише частку успіху, а й токени — саме вони зазвичай і пояснюють різницю.
7. **Довірчі інтервали (30 хв, ауд.).** Інтервали Вілсона — описово для кожної конфігурації; висновок робиться за інтервалом *парної різниці* (`bootstrap`, одиниця ресемплінгу — задача). *Письмово відповісти:* накриває інтервал різниці нуль чи ні. Відповідь «накриває» — нормальний результат роботи.
8. **Суддя і його калібрування (50 хв, сам.).** Розмітити 20 випадків вручну, порахувати узгодженість. *Перевірка:* звітується каппа, а не сира частка збігів.
9. **Траси (40 хв, сам.).** JSONL за конвенціями OpenTelemetry. *Перевірка:* за трасою видно, на якому саме кроці зламалася конкретна задача.

Крок 5 має спокусу: зробити `supervisor` «розумнішим», давши йому кращий промпт. Тоді порівняння втрачає сенс. Різнитися має рівно одна річ — топологія.

Завдання

1. Реалізувати МСР-сервер лише для читання з двома інструментами над локальними даними (наприклад, пошук у корпусі з лабораторної 2 і читання довідкової таблиці). Транспорт `stdio`, схеми аргументів і результату описані явно.
2. Реалізувати клієнта, що під'єднується до сервера, отримує перелік інструментів і викликає їх. Додати аудит-лог: час, інструмент, аргументи, результат, тривалість.
3. Написати тести схем на двох рівнях. Виклик з відсутнім обов'язковим полем і з полем неправильного типу не проходить валідацію схеми і має завершуватися протокольною помилкою JSON-RPC, а не виконанням з дефолтами. Виклик із семантично хибними даними (неіснуюча дата, невідомий ідентифікатор) схему проходить і має повертати результат із `isError: true` та повідомленням, з якого впливає наступна дія. Окремим тестом перевірити виклик невідомого інструмента.

4. Скласти набір із 40 задач над цими даними з відомими правильними відповідями. Набір фіксується до експериментів.
5. Реалізувати дві конфігурації з однаковими інструментами і однаковою моделлю: одиночний агент і supervisor з двома виконавцями (наприклад, пошук і перевірка цитат).
6. Прогнати обидві конфігурації по три рази на всіх 40 задачах. Виміряти: частку розв’язаних, pass^3 , точність вибору інструментів, токени й затримку на задачу, кількість помилок координації.
7. Порахувати інтервали Вілсона для часток успіху кожної конфігурації (описово) і 95 % довірчий інтервал парної різниці між конфігураціями (bootstrap із ресемплінгом за задачами, три прогони всередині задачі усереднюються). Письмово відповісти, чи накриває інтервал різниці нуль, і чому перекриття маргінальних інтервалів висновком не є.
8. Реалізувати суддю на основі моделі для оцінки якості відповіді, розмітити вручну 20 випадків і порахувати узгодженість судді з людською міткою. Якщо узгодженість низька, змінити інструкцію судді і повторити.
9. Зберігати траси у форматі JSONL з атрибутами за конвенціями OpenTelemetry: система, операція, вхідні й вихідні токени, ідентифікатор сесії. Показати, як за трасою відновлюється шлях однієї невдалої задачі.
10. Здати: код сервера і клієнта, набір задач, таблицю порівняння двох конфігурацій із довірчими інтервалами, результати калібрування судді, приклад траси і висновок до 200 слів із явним рішенням, чи виправдана мультиагентна конфігурація.

Контрольні запитання

1. Як протокол перетворює $M \times N$ інтеграцій на $M + N$?
2. Чим ресурс відрізняється від інструмента в термінах ініціативи?
3. Чому підключення MCP-сервера не є перевіркою його безпеки?
4. У чому різниця між $\text{pass}@k$ і pass^k і чому для надійності важлива друга?
5. Конфігурація А розв’язала 31 задачу з 40, конфігурація В — 33. Що з цього впливає?
6. Які три зсуви властиві судді на основі моделі?
7. Навіщо калібрувати суддю, якщо він і так узгоджується з інтуїцією?
8. Які метрики фіксують координаційну вартість мультиагентної системи?
9. Чому набір задач і конфігурації мають бути однаковими при порівнянні?
10. Що саме треба записати у трасу, щоб через тиждень відтворити невдалий прогін?

Література

1. Model Context Protocol. Architecture і Server: Tools.
2. Zheng L. et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. NeurIPS, 2023.
3. Anthropic. How We Built Our Multi-Agent Research System. Engineering blog, 2025.
4. Cognition. Don’t Build Multi-Agents. 2025.
5. Cemri M. et al. Why Do Multi-Agent LLM Systems Fail? arXiv:2503.13657, 2025.

6. OpenTelemetry. Semantic Conventions for Generative AI.
7. DeBenedetti E. et al. AgentDojo: A Dynamic Environment to Evaluate Attacks and Defenses for LLM Agents. NeurIPS, 2024.
8. Yao S. et al. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv:2406.12045, 2024.

Походження шаблону

Макет адаптовано з Laboratory report template for Basic Chemistry at Stockholm University, Viktor Bengtsson, LPPL 1.3c: article, Times-подібна гарнітура, широкі поля, fancyhdr, booktabs.